

UACM

Universidad Autónoma
de la Ciudad de México

Nada humano me es ajeno

Academia de Informática Cuauhtepac
Ingeniería de Software
Construcción y Evolución de Software
Profesor Emmanuel Vite Sevilla

Nombre de la práctica: Programa uno

Número: 2

Alumno(s)/Equipo:

Axel Javier Rios Sánchez

Jesús Josué Suarez López

Evelin Maldonado Espinosa

Mauricio Charbel Chavez Galindo

Fecha: 27/02/2026



Reporte de Evolución de Diseño: Del Prototipo a la Versión Funcional

El diseño inicial presentaba limitaciones críticas en tres áreas fundamentales: la experiencia de usuario (UX), la integración de reglas lógicas y la robustez del código.

En cuanto a la interfaz, el diseño original sufría de una visualización vertical ineficiente; al imprimir cada carta una debajo de otra, el jugador perdía el contexto de la mesa y de su propia mano debido al exceso de "scroll". La Versión 2 soluciona esto mediante una renderización horizontal y el uso de colores ANSI, lo que permite identificar cartas por color de forma instintiva y ver la mano completa en bloques compactos, mejorando drásticamente la jugabilidad en consola.

Desde la perspectiva de las reglas del juego, el primer diseño era un sistema simplificado que solo reconocía números. Carecía de la esencia del UNO: las cartas de acción y los comodines. La Versión 2 introduce una lógica de efectos donde cartas como "+2", "Salto" o "+4" alteran activamente el flujo de turnos y el tamaño de las manos de los oponentes. Además, el diseño inicial fallaba al no validar la primera carta de la mesa, mientras que la versión actual garantiza un inicio legal (una carta numérica no negra) y gestiona correctamente el estado del "color actual" cuando se juegan comodines.

Finalmente, la robustez y estructura del código han dado un salto cualitativo. El diseño inicial era vulnerable a entradas de usuario inválidas (como letras en campos numéricos), lo que provocaba cierres inesperados. La Versión 2 implementa métodos de validación como `leerOpcion`, que encapsula el manejo de excepciones. Asimismo, la lógica de juego se ha desprendido del método `main` para distribuirse en métodos especializados (`procesarJugada`, `esValida`), facilitando el mantenimiento y permitiendo que la CPU tome decisiones más coherentes, como elegir un color estratégicamente tras usar un comodín.

INFORME DE CONTROL DE CALIDAD (QA) - PROYECTO UNO v2.0

Documento: Evaluación de Evolución de Software

Responsable: Evelin Maldonado Espinosa

Equipo de Desarrollo: Axel Javier Rios Sánchez, Jesús Josué Suarez López, Evelin Maldonado Espinosa, Mauricio Charbel Chavez Galindo

Estado del Proyecto: Beta Estable (con observaciones de lógica)

1. MATRIZ DE CUMPLIMIENTO Y ERRORES

Esta tabla detalla qué requisitos de la "Versión Evolucionada" se cumplen y qué errores técnicos presenta la implementación actual respecto a las reglas oficiales.

Requisito	Estado	Error / Observación Técnica	Gravedad
Cartas Especiales	☑ OK	Se integran correctamente: +2, salto, rev, +4 y comodín.	-
Efecto Reversa	⚠ Error	En aplicarEfecto, el cambio de dirección solo funciona bien con 3+ jugadores. Con 2 jugadores, la lógica de dirección *= -1 suele actuar como un "Salto", pero tu código fuerza un avanzar(), lo que puede duplicar el salto.	Media
Efecto Salto / +2 / +4	✗ Error	Doble Salto de Turno: Tu método aplicarEfecto ya llama a avanzar(), pero el bucle principal en Main vuelve a llamar a juego.avanzar() al final. Esto causa que se salte a dos personas.	ALTA
Regla "UNO"	☑ OK	Implementada mediante entrada por consola. El castigo de +2 cartas es correcto según el estándar.	-
Penalización Robo	⚠ Lógica	El requisito pide "Penalización por no poder jugar tras robar". En tu código, el jugador puede elegir jugar la robada, pero si no puede, el turno pasa siempre. Falta un mensaje claro de "No puedes jugar, pierdes el turno".	Baja
Manejo de Color	⚠ Lógica	Al usar un Comodín, cambias colorActual, pero la mesa.valor sigue siendo la de la carta anterior. Esto puede confundir la validación de la siguiente jugada.	Media

2. REVISIÓN DE REGLAS OFICIALES (MATTEL)

Se contrastó el software con el reglamento oficial de Mattel para verificar la fidelidad de la simulación:

- Validación de "UNO": El sistema solicita la confirmación *después* de procesar la jugada. Para mayor rigor técnico, esta validación debería ocurrir de forma simultánea al descarte.
- Gestión de Cartas Negras: La implementación permite jugar el comodín +4 de manera libre. Según el reglamento oficial, esta carta solo debe usarse si el jugador no posee cartas del color en mesa.
- Mecánica de Robo: El flujo actual de "robar una vez y pasar" es correcto y se alinea con las reglas estándar de competencia.

3. EVIDENCIAS DE HALLAZGOS (CAPTURAS DE CÓDIGO)

```
}  
  
if(actual.mano.size()==1){  
  
    System.out.println("Di UNO:");  
  
    String u=sc.nextLine();  
}
```

FOTO 1: Evidencia de conflicto en el control de flujo. Se observa que el método aplicarEfecto() ejecuta un avance de turno (avanzar()) de forma redundante con el ciclo principal en la clase Main. Esta falta de sincronización provoca un "doble salto", donde se omite el turno de dos jugadores en lugar de uno, invalidando la mecánica oficial de las cartas de acción.

```
Carta c=actual.mano.get(idx);  
  
if(!juego.jugarCarta(actual,c))  
    System.out.println("Carta inválida");
```

FOTO 2: Análisis de caso de borde (Edge Case) para la carta Reversa. En una configuración de dos jugadores, el algoritmo no conmuta el sentido del juego de manera efectiva. Al sumar el avance manual del método con el avance automático del ciclo de juego, el turno retorna sistemáticamente al emisor, imposibilitando la alternancia de turnos reglamentaria de Mattel.

```

void aplicarEfecto(Carta c){
    if(c.valor.equals("rev")){
        if(jugadores.size()==2){
            avanzar();
        }else{
            direccion*=-1;
        }
    }
}

```

FOTO 3:Auditoría de reglas de negocio. La validación del estado de la mano (size == 1) se ejecuta de forma reactiva tras el procesamiento de la jugada. Esta implementación permite que la carta llegue a la mesa antes de verificar el "grito de UNO", facilitando que el usuario corrija su olvido y debilitando la penalización estratégica del juego original.

```

}

if(actual.mano.isEmpty()){
    System.out.println("\nGANÓ "+actual.nombre);
    jugando=false;
}

juego.avanzar();

```

FOTO 4:Fallo en la gestión de entradas inválidas del usuario. Se identifica que tras una selección de carta no válida (incumplimiento de color o número), el sistema imprime la alerta pero no reinicia el sub-ciclo de selección. Esto deriva en una "muerte de turno" accidental, donde el jugador pierde su oportunidad de jugar sin haber realizado un movimiento válido.⁴

```

void aplicarEfecto ( Carta c ) {

    si ( c . valor . equals ( "rev" )) {

        if ( jugadores . tamaño ( ) == 2 ) {
            avanzar ( ) ;
        } demás {
            dirección *= -1 ;
        }
    }
}

```

FOTO 5: Inconsistencia visual en el objeto Mesa (UI/UX). Al utilizar un comodín, el sistema actualiza la variable lógica colorActual, pero el objeto mesa mantiene sus atributos visuales originales. Esto genera una discrepancia donde el dibujo de la carta no coincide con el estado lógico del juego, pudiendo inducir al error en las jugadas posteriores.

4. CONCLUSIÓN Y DIAGNÓSTICO

Tras concluir el ciclo de pruebas sobre la Versión 2.0 (Evolución), se determina que el software ha integrado con éxito las nuevas mecánicas de juego. El programa demuestra una arquitectura sólida en la gestión de objetos y una interfaz de consola funcional.

No obstante, se ha detectado deuda técnica en la sincronización de turnos y en la actualización visual de la mesa. El proyecto cumple con los criterios de evaluación académica, entregando un producto funcional que sirve como base para futuras implementaciones de interfaz gráfica.

Veredicto: ☐ APROBADO CON OBSERVACIONES TÉCNICAS

NADA HUMANO ME ES AJENO

CÓDIGO:

```
import java.util.*;

class Carta{

    String color;
    String valor;

    static final String RESET="\u001B[0m";
    static final String ROJO="\u001B[31m";
    static final String VERDE="\u001B[32m";
    static final String AMARILLO="\u001B[33m";
    static final String AZUL="\u001B[34m";
    static final String BLANCO="\u001B[37m";

    public Carta(String color,String valor){
        this.color=color;
        this.valor=valor;
    }

    String colorANSI(){
        switch(color){
            case "Rojo": return ROJO;
            case "Verde": return VERDE;
            case "Azul": return AZUL;
            case "Amarillo": return AMARILLO;
            default: return BLANCO;
        }
    }
}
```

```

public String[] dibujar(){

    String c=colorANSI();

    return new String[]{
        c+"-----"+RESET,
        c+"| "+String.format("%-8s",valor)+" |"+RESET,
        c+"|      |"+RESET,
        c+"| "+String.format("%-8s",color)+" |"+RESET,
        c+"|      |"+RESET,
        c+"| "+String.format("%8s",valor)+" |"+RESET,
        c+"-----"+RESET
    };
}
}

```

```

class Baraja{

```

```

    List<Carta> cartas=new ArrayList<>();

```

```

    public Baraja(){
        crear();
        Collections.shuffle(cartas);
    }

```

```

    void crear(){

        String[] colores={"Rojo","Verde","Azul","Amarillo"};

        for(String color:colores){

```



```

        cartas.add(new Carta(color,"0"));

        for(int i=1;i<=9;i++){
            cartas.add(new Carta(color,""+i));
            cartas.add(new Carta(color,""+i));
        }

        for(int i=0;i<2;i++){
            cartas.add(new Carta(color,"+2"));
            cartas.add(new Carta(color,"salto"));
            cartas.add(new Carta(color,"rev"));
        }
    }

    for(int i=0;i<4;i++){
        cartas.add(new Carta("Negro","+4"));
        cartas.add(new Carta("Negro","comodin"));
    }
}

Carta repartir(){

    if(cartas.isEmpty()) return null;

    return cartas.remove(cartas.size()-1);
}

}

class Jugador{

```

```

String nombre;
List<Carta> mano=new ArrayList<>();

public Jugador(String nombre){
    this.nombre=nombre;
}

void recibir(Carta c){
    if(c!=null) mano.add(c);
}

void verMano(){
    System.out.println("\nMano de "+nombre);

    int arriba=Math.min(4,mano.size());
    int abajo=Math.min(3,Math.max(0,mano.size()-4));

    mostrarFila(0,arriba);
    mostrarFila(4,4+abajo);
}

void mostrarFila(int inicio,int fin){

    if(inicio>=mano.size()) return;

    for(int i=inicio;i<fin;i++)
        System.out.print("  ["+i+"]  ");

```

```

        System.out.println();

        for(int linea=0;linea<7;linea++){

            for(int i=inicio;i<fin;i++){

                String[] d=mano.get(i).dibujar();
                System.out.print(d[linea]+" ");

            }

            System.out.println();
        }
    }
    System.out.println();
}

```

```

class Juego{

    List<Jugador> jugadores=new ArrayList<>();
    Baraja mazo=new Baraja();

```

```

    Carta mesa;
    String colorActual;

```

```

    int turno=0;
    int direccion=1;

```

```

    Random random=new Random();
    Scanner sc=new Scanner(System.in);

```

```
void registrar(String nombre){  
  
    jugadores.add(new Jugador(nombre));  
    jugadores.add(new Jugador("CPU"));  
}
```

```
void repartir(){  
  
    for(Jugador j:jugadores)  
        for(int i=0;i<7;i++)  
            j.recibir(mazo.repartir());  
    boolean cartaValida=false;  
  
    while(!cartaValida){  
        mesa=mazo.repartir();  
        if(mesa.valor.matches("\\d+"))  
            cartaValida=true;  
        else{  
            mazo.cartas.add(mesa);  
            Collections.shuffle(mazo.cartas);  
        }  
    }  
  
    colorActual=mesa.color;  
}
```

```

boolean jugarCarta(Jugador j, Carta c){

    if(c.color.equals(colorActual)
    || c.valor.equals(mesa.valor)
    || c.color.equals("Negro")){

        j.mano.remove(c);
        mesa=c;

        if(!c.color.equals("Negro"))
            colorActual=c.color;
        else{
            if(!j.nombre.equals("CPU")){
                int op=0;
                while(true){

                    System.out.println("Elige color:");
                    System.out.println("1 Rojo");
                    System.out.println("2 Verde");
                    System.out.println("3 Azul");
                    System.out.println("4 Amarillo");

                    try{
                        op=Integer.parseInt(sc.nextLine());
                        if(op>=1 && op<=4) break;
                    }catch(Exception e){}
                }
            }
        }
    }
}

```

```

        System.out.println("Opción inválida.");
    }

    switch(op){
        case 1: colorActual="Rojo"; break;
        case 2: colorActual="Verde"; break;
        case 3: colorActual="Azul"; break;
        case 4: colorActual="Amarillo"; break;
    }

    }else{
        String[] colores={"Rojo","Verde","Azul","Amarillo"};
        colorActual=colores[random.nextInt(colores.length)];
        System.out.println("CPU eligió color: "+colorActual);
    }
}

    aplicarEfecto(c);

    return true;
}

    return false;
}

void aplicarEfecto(Carta c){

```

```

        if(c.valor.equals("rev")){

            if(jugadores.size()==2){
                avanzar();
            }else{
                direccion*=-1;
            }
        }
    }

    if(c.valor.equals("salto"))
        avanzar();

    if(c.valor.equals("+2")){
        avanzar();
        Jugador j=jugadores.get(turno);

        j.recibir(mazo.repartir());
        j.recibir(mazo.repartir());
    }

    if(c.valor.equals("+4")){

        avanzar();
        Jugador j=jugadores.get(turno);

        for(int i=0;i<4;i++)
            j.recibir(mazo.repartir());
    }
}

```

```

void avanzar(){

    turno=(turno+direccion+jugadores.size())%jugadores.size();

}
}

```

```

public class Main{

```

```

    static Scanner sc=new Scanner(System.in);

```

```

    public static void main(String[] args){

```

```

        System.out.println("Introduce tu nombre:");

```

```

        String nombre=sc.nextLine();

```

```

        Juego juego=new Juego();

```

```

        juego.registrar(nombre);

```

```

        juego.repartir();

```

```

        boolean jugando=true;

```

```

        while(jugando){

```

```

            Jugador actual=juego.jugadores.get(juego.turno);

```

```

            System.out.println("\nCarta en mesa:");

```

```

            for(String l:juego.mesa.dibujar())

```

```

                System.out.println(l);

```



```
System.out.println("Color actual: "+juego.colorActual);
```

```
if(!actual.nombre.equals("CPU")){
```

```
    actual.verMano();
```

```
    int opcion=0;
```

```
    while(true){
```

```
        System.out.println("1. Jugar carta");
```

```
        System.out.println("2. Robar");
```

```
        try{
```

```
            opcion=Integer.parseInt(sc.nextLine());
```

```
            if(opcion==1 || opcion==2) break;
```

```
        }catch(Exception e){}
```

```
        System.out.println("Opción inválida.");
```

```
    }
```

```
    if(opcion==1){
```

```
        int idx;
```

```
        while(true){
```

```
            System.out.println("Indice carta:");
```

```

try{

    idx=Integer.parseInt(sc.nextLine());

    if(idx>=0 && idx<actual.mano.size())
        break;

}catch(Exception e){

    System.out.println("Índice inválido.");
}

Carta c=actual.mano.get(idx);

if(!juego.jugarCarta(actual,c))

    System.out.println("Carta inválida");
}

else{
    Carta robada=juego.mazo.repartir();

    System.out.println("\nRobaste:");

    for(String l:robada.dibujar())
        System.out.println(l);

    while(true){

```

```

        System.out.println("¿Quieres jugarla? (s/n)");

        String r=sc.nextLine().toLowerCase();

        if(r.equals("s")){

            if(!juego.jugarCarta(actual,robada))
                actual.recibir(robada);

            break;
        }

        if(r.equals("n")){
            actual.recibir(robada);
            break;
        }

        System.out.println("Respuesta inválida.");
    }

    NADA HUMANO ME ES AJENO

    if(actual.mano.size()==1){

```

```

        System.out.println("Di UNO:");

```

```

        String u=sc.nextLine();

```

```

        if(!u.equalsIgnoreCase("UNO")){

```

```

            System.out.println("Penalización +2");

```

```

        actual.recibir(juego.mazo.repartir());
        actual.recibir(juego.mazo.repartir());
    }
}
}

```

```

else{

```

```

    boolean jugo=false;

```

```

    for(Carta c:new ArrayList<>(actual.mano)){

```

```

        if(juego.jugarCarta(actual,c)){

```

```

            System.out.println("\nCPU jugó "+c.color+" "+c.valor);

```

```

            jugo=true;

```

```

            break;

```

```

        }

```

```

    }

```

```

    if(!jugo){

```

```

        Carta robada=juego.mazo.repartir();

```

```

        actual.recibir(robada);

```

```

        System.out.println("\nCPU roba carta");

```

```

    }

```

```

}

```

```
if(actual.mano.isEmpty()){  
  
    System.out.println("\nGANÓ "+actual.nombre);  
    jugando=false;  
}  
  
juego.avanzar();  
}  
}
```

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO